BVP INDIA · PORTFOLIO AI PLAYBOOK

Part II

Compounding the Advantage

From Loop to Native

The L3 to L4 transition for BVP portfolio companies. Multi-agent architectures, eval discipline, self-improvement loops, institutional memory, and the org structure that makes all of it durable.

AUTHORED BY

Ishan Parikh · Bessemer Venture Partners India

INTENDED AUDIENCE

Portfolio companies operating at L3 (Loop) on the BVP AI fluency ladder, with Part I foundations stable. If Part I (Foundations) is not yet installed, start there. Part II assumes Claude Enterprise, MCP connectors, Projects per function, a skills library, and the Part I guardrails are already running.

May 2026 · Portfolio Internal

Contents

The compounding stack

Multi-agent architectures

The eval factory

Recipe 4 · The performance marketing agent team

Recipe 5 · The dormant customer recovery team

Recipe 6 · Per-customer agents

Recipe 7 · Synthetic persona testing

Recipe 8 · Overnight self-improvement loops

Recipe 9 · Institutional memory as a queryable layer

Recipe 10 · The engineering orchestrator

Voice-native operations

Long-horizon agents and computer use

The org chart of an AI-native company

Pricing for AI-native B2B

The sequence

Appendix

The compounding stack

Part I gets a company from L1 to L2, with one foot in L3. That alone produces real results. From our portfolio observations: creative output roughly doubles within the first cycle of disciplined use; support agents resolve a meaningful majority of tickets without a human (typically around 80 percent for the better implementations, measured as the share of incoming tickets closed without human reassignment); voice outreach reaches Tier 2 and Tier 3 cities at a fraction of the cost of a call centre (measured as fully-loaded cost per outbound contact versus a call-centre baseline, portfolio observation). A founder who shipped Part I has already opened a gap on every competitor who has not.

Part II is about what happens after that gap opens. The competitor who is behind can buy the same Claude Enterprise seats and wire the same MCP connectors. What they cannot buy is a year of evals, a library of synthetic personas built from real conversations, a self-improvement loop that has been quietly rewriting prompts on a schedule, and an institutional memory layer that every new agent queries from day one.

Reimagining the company as recursive AI loops

Most companies today are organised like a Roman legion: nested hierarchies with humans acting as the conduit for information flowing up and down. AI breaks that model. A company that has installed Part I infrastructure is already partially free of it. A company that completes Part II rebuilds itself as a set of recursive self-improving AI loops.

Each loop has the same five layers. Build them right and the loop improves itself while the team sleeps. Get any layer wrong and the loop quietly fails until someone notices.

LAYER	WHAT IT IS	EXAMPLE
Sensor	Inputs from the outside world the loop reads.	Customer emails, support tickets, product telemetry, cancellations, code changes.
Policy	Rules about what the agent can do, what needs human approval, what it must log.	The action-irreversibility ladder from Part I, applied to this specific loop.
Tool	Deterministic APIs and skills the agent can call.	Database queries, calendar reads, MCP connector calls, custom code in the Claude Code skills library.
Quality gate	Evals, safety filters, human review for high-risk cases.	The golden dataset from Chapter 3. Shadow mode. The promotion gate.

Learning	The mechanism that picks up what did not work and feeds it back to the top.	The monitoring agent that watches failures and proposes fixes. Recipe 8 covers this in full.

Every recipe in this part of the book is one of these loops. Recipe 4 (performance marketing) is a loop. Recipe 5 (dormant recovery) is a loop. Recipe 8 (self-improvement) is the loop that improves other loops.

> **WHAT L4 ACTUALLY LOOKS LIKE**
>
> At an India-native OTT platform in our portfolio, every team member ships their own agents. The marketing lead built an agent that auto-tunes ad targeting across geographies. A product manager built a game where users play against an AI character from one of the company's own shows. A CX lead built a synthetic-persona test bed that runs before any prompt change goes live. Same-day implementation cycles. Agents that build other agents. That is L4. The four mechanisms below are what makes it possible.

Mechanism one: multi-agent architectures

At L3, the company has a few production agents. They work, but they are isolated. The support agent does not know what the marketing agent is doing. The dormant-recovery agent and the per-customer agent fight each other for the same outbound slot.

L4 introduces coordinator patterns. A top-level agent that knows which subagent owns which decision, what each one needs from the others, and when to escalate to a human. Chapter 2 covers the three architectures that work in production and the one that does not.

Mechanism two: eval discipline

At L2, every prompt change is a manual decision. Someone tries the new version on five examples and decides if it works. At L4, every prompt change runs through a golden dataset before it touches a customer. The regression bar applies automatically. Shadow mode runs for a defined window before write access is restored. Continuous evals run after deployment.

This is the single mechanism that separates production agents from demos. Chapter 3 covers how to build it.

FROM A PORTFOLIO COMPANY WE WORK WITH

The founder of one of our B2B SaaS portcos made the point clearly in our conversation. "You get the best outcome with a deterministic flow and AI on top of it, not an entirely AI-driven flow." Production AI needs a backbone that does not break. The eval factory in Chapter 3 is the infrastructure that makes a deterministic flow with AI on top actually deterministic.

Mechanism three: self-improvement loops

Most agents ship and are still running the original prompts long after deployment. The team that built them moved on. Customer behaviour shifted. The competitive set changed. The agent has not noticed any of it.

At L4, an automated process tries variants of prompts, scoring functions, and tool calls against quality metrics and keeps the winners. The pattern reference is Andrej Karpathy's autoresearch framework. Chapter 8 covers the implementation, including the safety rules that prevent an agent from rewriting its own behaviour into something the founder would not have approved.

Mechanism four: institutional memory

Most companies accumulate knowledge: customer research, campaign reports, competitor intelligence, SOPs, founder memos. At L3, this lives in Notion and Drive. Agents can search it but the results are stale, contradictory, and missing context. At L4, every entity that matters (customer, competitor, SKU, campaign) has two things in the knowledge layer. A compiled current truth, rewritten by an agent whenever new evidence arrives. An append-only evidence timeline, never edited.

The pattern reference is Garry Tan's open-source GBrain. Chapter 9 covers how to implement this in Notion, Postgres, or a hybrid, with Claude Code maintaining it on a heartbeat. Every other agent queries this layer at runtime via MCP.

Skills are how the mechanisms compound

Anthropic's framing in the Founder's Playbook is worth lifting directly. A skill is a reusable workflow written once in plain English, saved to a Git repository, callable by name from any conversation. Anyone in the company can write one. Anyone can improve one. The marketer who writes a great skill in the morning makes every other person on the team better at that task by the afternoon.

Every loop in this part of the book lives in a skill. Recipe 4's performance marketing team is a top-level skill called performance-marketing-team. Recipe 8's self-improvement runner is a skill called overnight-research. Skills accumulate. At L2, the company has ten skills. At L3, fifty. At L4, hundreds. The skills library is the moat. Nothing else compounds the same way.

Why these mechanisms, in this order

The mechanisms build on each other. Multi-agent architectures (mechanism one) need eval discipline (mechanism two) to keep the subagents honest. Self-improvement loops (mechanism three) need eval discipline as the quality metric and a multi-agent architecture as the target. Institutional memory (mechanism four) is what every other mechanism queries for context.

Skip mechanism two and the rest of the stack runs on faith. Skip mechanism four and every agent reinvents what the company already knows. The order matters: install eval discipline first, multi-agent architectures next, institutional memory third, self-improvement last. Recipes 4 through 9 sequence this way.

Multi-agent architectures

At L2, the question is whether the agent works. At L3, the question is how many agents work together. The three architectures below cover almost every L3-to-L4 use case in the BVP consumer and SaaS portfolio. Pick the right one for the task. The wrong choice is the single most common failure mode at this stage.

Architecture one: single agent with tools

One Claude instance. Multiple tool calls. The agent decides which tool to use in what order. Examples: the AI customer support agent from Part I, the Wispr Flow dictation flow for executives, a sales-prep agent that pulls the CRM record, the last six emails, and the most recent customer call transcript.

Use single-agent architectures when the task is bounded, the context fits in a single window, and the failure modes are well understood. Do not use multi-agent architectures for tasks a single agent can handle. Coordination overhead is real.

Architecture two: coordinator with subagents

One top-level agent that decomposes a problem into subtasks, dispatches each subtask to a specialised subagent, then synthesises the results. This is the pattern that powers Recipes 4 and 5 in this book. The Anthropic multi-agent research framework is the public reference implementation.

Each subagent has its own system prompt, its own tools, its own evaluation rubric, and ideally its own narrowly-scoped knowledge base. The coordinator does not need to know how the subagent works internally. It needs to know what to send in and what to expect back.

> **THE FAMOUS PERSONA TRICK**
>
> When defining each subagent's personality, give it a famous character's reference (from TV, movies, public figures, or anywhere else with a recognisable archetype). Do not write "you are a marketing optimiser." Write "you are Don Draper from Mad Men season five, data-driven but creative-respecting." Large language models train on enormous quantities of public data. They understand character archetypes more deeply than they understand job titles. A famous persona compresses thousands of training examples of that character's decision style, risk tolerance, and communication norms into two or three words. Better reasoning, free.

Architecture three: swarm

Many agents, no central coordinator, each running on its own trigger or schedule. They share state through a common knowledge layer (the institutional memory from Chapter 9) and communicate by writing to it rather than calling each other directly.

Swarm architectures are appropriate when the agents do not need to coordinate in real time and the work is naturally parallel. The per-customer agent fleet in Recipe 6 is a swarm: ten thousand agents, each managing one customer, none of them aware of the others, all querying and writing to the same memory layer.

Swarms are also the architecture that fails most often at the start. The institutional memory layer has to be in place first. Without it, agents step on each other constantly.

Architecture four: do not use

Agents that call agents directly in a chain, without a coordinator and without a shared memory layer. This is the architecture that looks elegant in a tutorial and disastrous in production. Errors compound. Latency stacks. Debugging is nearly impossible because the input to agent C depends on agent B which depends on agent A and any of them might be on a stale prompt.

If a workflow seems to require a chain of agents, the right move is to convert it to a coordinator pattern with explicit handoffs, or to a swarm pattern with explicit reads and writes to the memory layer. Direct agent-to-agent calls are an antipattern.

A concrete failure mode. A campaign-brief agent writes to stdout. A copy agent reads from it. The copy agent picks up the prior run's brief. The campaign launches with last month's positioning. The post-mortem takes three days. A coordinator with explicit handoffs would have caught this in the first invocation.

The orchestration choice

Four real options at L3-L4. Pick one. Do not run more than two in production simultaneously.

ORCHESTRATOR	BEST FOR	TRADE-OFF
Claude Code with the Agent SDK	Single agents and coordinator patterns. Plan Mode for safety.	Less mature multi-day workflow tooling. Built for engineering teams.
n8n or Temporal	Long-running multi-step workflows that span days. Stateful retries.	Operational overhead. Worth it only when the workflow genuinely spans days.

LangGraph	Complex graph-shaped agent flows with conditional branching.	Steeper learning curve. Verbose for simple cases.
Custom (in-house)	When the team has a strong engineering culture and wants full control.	You own everything. Choose this only if the engineering function is L4 itself.

If you are not sure whether to use Temporal: you are not ready for Temporal. The Agent SDK plus a Postgres state table handles every recipe in this book.

How the recipes interact

The recipes in this part are not independent. Wiring them together is the difference between L3 and L4, and the wiring has rules.

Recipe 4's Campaign Builder calls the Recipe 1 creative factory. Wire them together only after confirming Recipe 1's rate limits can handle the volume.

Recipe 5 and Recipe 6 may both target the same customer. Per-customer agents (Recipe 6) take priority for any customer above the LTV threshold. Build a simple outbound lock so Recipe 5 skips customers already contacted by Recipe 6 in the current cycle.

Recipe 8 runs overnight on production prompts. Mark each skill file with loop-eligible: true or loop-eligible: false. The runner respects the flag.

Recipe 10 is the only recipe that writes to GitHub and Jira. Keep those connector scopes locked to the engineering orchestrator's service account.

> **BORROWED INTELLIGENCE · THE MULTI-MODEL PATTERN**
>
> One of our B2B SaaS portcos runs a multi-model dispatch layer that routes each task to the right model based on complexity. Claude for reasoning. OpenAI for some workflows. An in-house OSS LLM on their own servers for sensitive tasks. The pattern is portable. Any portco at L3 with cost or compliance concerns should adopt it. It also reduces single-vendor concentration risk, which matters more than founders generally appreciate until the day the primary model provider has a four-hour incident.

Observability is not optional

Every multi-agent system needs an observability layer. Three things must be logged for every subagent invocation: the input it received from the coordinator, the tool calls it made, and the output it returned. Stored for ninety days minimum. Searchable by agent name, user identity, and date range.

Helicone, Langfuse, and Arize all do this well. Pick one. If the engineering function prefers to build their own, the spec is short: write every agent call to a single Postgres table with the columns above plus a model version, a prompt commit hash, and a duration. That table is your incident response tool and your eval input. Both Chapters 3 and 8 assume this exists.

The eval factory

The single thing separating production agents from demos. An AI-native B2B SaaS company in the BVP portfolio (publicly documented via the Bessemer Atlas case study) described their internal eval framework as the "machine for building the machine." Backtesting against historical data. Simulated user behaviour. Large-scale A/B tests validating every change against resolution rate, customer satisfaction, and hallucination frequency. They built this before their agent ever touched a customer. It is what let them be a launch partner with the frontier model confidently.

Every portfolio company crossing from L3 to L4 needs the same infrastructure. This chapter covers what it actually contains.

Step one: build the golden dataset

For every agent in production, pull a representative sample of real inputs from the last ninety days. The CX agent gets a sample of tickets. The marketing agent gets a sample of campaign briefs. The voice agent gets a sample of call transcripts.

Three categories. First, the common cases. The seventy percent of inputs that drive eighty percent of outcomes. Aim for fifty examples here. Second, the edge cases. The inputs that broke the agent in the past, or that a thoughtful human flagged as risky. Aim for twenty examples. Third, the adversarial cases. Inputs deliberately designed to trip the agent, including prompt injection attempts. Aim for ten.

Each example needs three things: the input, the expected behaviour described in plain English (not the expected output, the expected behaviour), and the score rubric. The technical owner writes the rubric in collaboration with the function lead. This takes a working week. Do not skip it.

When building golden datasets from real user data, strip fields the agent does not need before storing the dataset. Eval data retains and expires on the same schedule as the source production data. No exceptions. Set the dataset’s retention to match the operational retention of the source system, and avoid storing raw payloads beyond that window. This is operational discipline, not legal compliance; the legal team writes the contracts, the technical owner writes the retention rules.

Step two: pick the judge

For each example, you need an automated way to decide if the agent's response was acceptable. Three options.

- Hard-coded checks. Best for structured outputs. Does the JSON validate? Does the response contain the required fields? Did the agent call the right tool? Cheap, fast, deterministic.
- LLM-as-judge. A second Claude call (typically Haiku for cost) that reads the input, the agent's response, and the rubric, and returns a score plus reasoning. Best for evaluating tone, brand voice, completeness. Requires a rubric the judge can apply consistently.
- Human review. Best for the adversarial set and for any sample where the LLM judge expresses uncertainty. The most expensive option, so reserve it for the cases that genuinely need a human eye.

Most production eval setups use all three. Hard-coded checks for structure. LLM-as-judge for quality. Human review for the long tail and for periodic recalibration of the judge itself.

Pin the judge's model version in your eval config. When you upgrade the judge, run both versions on 100 historical examples and confirm the delta is under 2 percentage points before switching.

Step three: set the promotion gate

An agent does not go to production until it passes the gate. The default gate, calibrated across the BVP portfolio engagements I have done:

- Common cases: 90 percent acceptable
- Edge cases: 70 percent acceptable
- Adversarial cases: 0 catastrophic failures (any prompt injection that succeeds, any PII leak, any action outside the irreversibility tier, is a hard stop)

These thresholds apply to text and conversation agents. For code, use Recipe 10's CI/CD gate instead.

When the prompt changes, all three gates run again. The regression bar is no drop of more than three percentage points on common or edge, and zero new catastrophic failures. If the new prompt fails the regression bar, it does not ship, even if it scores better overall.

Step four: shadow mode

The agent has passed the gate. It still does not get write access immediately. It runs in shadow mode for a defined window: every input that arrives gets routed both to the old version (which acts) and to the new version (which logs what it would have done). The technical owner samples the divergences on a regular cadence. Anything surprising blocks promotion.

Shadow mode is what catches the failure modes that golden datasets miss: the live distribution shift, the seasonal effect, the customer cohort the dataset under-represents. Configure the window length to fit the action's reversibility. A draft email might need a shorter window. A pricing change or a customer-facing announcement should run shadow for longer.

Step five: continuous evals after deployment

The eval factory does not stop running when the agent ships. Every day, a sample of production traffic gets scored by the same judge against the same rubric. The dashboard shows the trailing seven-day score and the trailing thirty-day score. When either dips below a threshold, an alert fires. This is how a regression that escaped the gate gets caught in production.

Two thresholds matter. The hard threshold: trailing score drops below 85 percent of the gated baseline. The agent rolls back automatically. The soft threshold: trailing score drops more than five points week over week. The technical owner gets paged.

> **THE EVAL FACTORY IS THE ORG'S COMPLIANCE POSTURE**
>
> An enterprise procurement team signing a multi-year contract will ask for this. SOC 2 auditors will ask for this. The DPDPA grievance officer (when one is assigned to the company) will ask for this. The same infrastructure that makes agents better also makes the company fundraiseable, sellable, and defensible. Build it because it makes the product better. The compliance posture is the second-order benefit.

Who owns the eval factory

The technical owner builds it once. Each function lead owns the golden dataset for their function's agents. Marketing lead owns the marketing eval set. CX lead owns the support eval set. Brand lead owns the creative eval set. Datasets grow with the company. New failure modes get added to the eval set as soon as they are discovered.

One of our portfolio companies made AI workflow creation a criterion in performance reviews for customer-facing roles. The only structural adoption mechanism we have seen in the BVP portfolio that gets non-engineering functions to genuinely own their agents. Lifting that pattern: maintaining the function's eval dataset, and the rate at which it expands, becomes a criterion in function leads' performance reviews. The ones who treat it as infrastructure rather than chore are the ones who will be running the L4 version of the company over the next few engagements.

Recipe 4 · The performance marketing agent team

Maturity level: L3. First recipe in this part that uses a coordinator pattern. The architecture is portable across every consumer portco doing meaningful Meta or Google spend.

Problem

Performance marketing has too many variables for any human to optimise. Channels (Meta, Google, quick commerce) times audiences times creatives times bid strategies produces more combinations than a team can test. Marketers focus on the ten percent of campaigns they can see. The other ninety percent runs on autopilot at whatever Customer Acquisition Cost the platforms decide to deliver.

Architecture

A coordinator pattern with five subagents. Build them inside Claude Code as a top-level skill called performance-marketing-team.

SUBAGENT	CADENCE	WHAT IT DOES
Analyst	Every 12 hours	Reads every active campaign, computes CAC, CPM, CTR, conversion against the targets, flags anomalies.
Optimizer	2-3x daily on Meta, every 2 days on Google	Proposes bid adjustments, audience changes, creative rotations. Writes recommendations to Slack.
Budget Allocator	Daily	Reallocates spend across active campaigns based on Analyst output and CAC:LTV by cohort.
Campaign Builder	On demand	Creates new campaigns end-to-end. Pulls creative from the Recipe 1 factory. Drafts copy. Sets targeting.
Chief	Weekly	Reviews everything the other four did. Flags risk. Writes the weekly summary the founder reads.

The famous persona trick, applied here

Give each subagent a famous character reference in the system prompt. The Analyst is Sherlock Holmes (sceptical, evidence-driven, never claims certainty without data). The Optimizer is Don Draper from Mad Men season five (decisive, willing to make moves competitors will not). The Chief is Margaret Thatcher (writes summaries in declarative prose, never hedges).

The reporting-only calibration phase

Critical. After deployment, the team does not have write access to the ad platforms. The Optimizer posts recommendations to a dedicated Slack channel. A human (the head of growth, or a senior marketer they designate) approves each one with a thumbs-up reaction. Every override the human applies becomes a new entry in the Marketing Project knowledge base. The portco decides how long this phase lasts. The signal that it should end is consistency: the team's recommendations match the human decision rate above 80 percent (measured as the share of agent proposals approved without modification by the human reviewer).

Once that consistency holds, grant write access to the lowest-risk actions: bid adjustments within plus or minus 15 percent of the prior value, audience refinements, creative rotations within the approved library. New campaign creation stays human-approved for longer. The portco decides when to widen scope based on the override rate and the function lead's confidence.

Eval framework

Three measurements run continuously. CAC at the cohort level on agent-managed spend, versus the manual baseline from the prior quarter, with statistical significance computed weekly. CAC:LTV ratio weighted for subscriber quality (an agent optimising for raw CAC will find ways to bring in low-value users). And the override rate: the percentage of agent recommendations the human reversed. An override rate that trends down over time is the signal the team is learning. An override rate that stays flat or grows is the signal something is wrong with the system prompts or the evals.

Owner

Head of growth or head of performance marketing. The technical owner builds the initial skill and configures the subagents. The growth lead owns the output, the targets, and the override decisions. Eventually the growth lead should be modifying subagent prompts themselves without involving the technical owner. That handoff is what makes this recipe an L4 capability rather than an L3 one. The pace at which the handoff happens is up to the portco.

What good looks like at L4

- The growth lead is shipping new subagents unaided. Examples from the portfolio: a creative tester that spawns variant campaigns, a competitor-watch agent that flags bid-strategy changes, a return-on-spend forecaster that pulls campaigns early if the projection drops below threshold.
- Override rate trending down across iteration cycles. Measured weekly as the share of agent recommendations the human reversed. A flat or rising rate is the signal something is wrong with the system prompts or the evals.
- Agent-managed share of bid adjustments above 70 percent, with humans handling the higher-stakes campaign creation. Measured directly from the ad-platform change logs.
- CAC on agent-managed spend at or below the manual baseline at statistical significance. Measured weekly with the prior-quarter baseline as the reference.
- Time the head of growth spends in the ad platforms directly: down by at least 30 percent from the pre-deployment baseline, measured via a self-reported time-allocation check before and after deployment.

Recipe 5 · The dormant customer recovery team

Maturity level: L3. Second coordinator pattern. Closely related to Recipe 4 but operates on retention rather than acquisition.

Set the inactivity threshold to 1.5x your average repurchase interval before wiring the trigger. The default of 30 days suits daily-use apps. Adjust for your category.

Problem

Every consumer brand has a long tail of dormant customers. Signed up, used the product once or twice, disappeared. The traditional playbook (email drip, discount, retargeting) works on maybe five percent. The rest need per-customer intelligence: the right channel, the right message, the right time. Templates cannot deliver that. Humans cannot scale to deliver it. Agents can.

Architecture

Coordinator pattern with four subagents. Triggered by a state change in the user table (most commonly, the user crosses thirty days of inactivity, but the recipe handles other triggers).

SUBAGENT	WHAT IT DECIDES
Channel Selector	Push, SMS, WhatsApp, voice. Pulls the customer's response history. Picks the channel they actually engage with.
Message Crafter	Drafts the personalised message. Pulls recent product engagement, content affinity, language preference. Uses brand voice from the Brand Project.
Timing Optimiser	Picks the day and hour. Analyses the customer's engagement pattern from history. Schedules accordingly.
Content Recommender	Picks the content or product offer. Pulls from the catalogue. Filters for what the customer has not yet engaged with.

The channel cost matrix

Critical input to the Channel Selector. India-specific costs as of mid-2026. The matrix is what stops the agent from spamming voice calls to every dormant user.

CHANNEL	COST PER SEND	USE WHEN
Push notification	Free	Customer opts in and engages with push historically.
SMS	~0.15	Push fails or customer does not have the app installed.
WhatsApp	~1	High-intent re-engagement attempt. Personalised. Templated approval required.
Voice call (AI)	5-20	Top-LTV cohort only. After lower-cost channels have been tried.

Cheapest first. Escalate if no response within the customer's typical response window. Cap by customer LTV. A customer worth 500 rupees in expected lifetime value does not justify a twenty-rupee voice call.

Cost data is mid-2026. Verify against your current vendor contracts before deploying the Channel Selector.

Eval framework

The primary metric is reactivation rate of the dormant cohort versus a no-intervention control group, measured at 7, 14, and 30 days after the first message. The control group is essential. Without it, the team will overstate the impact of every change. Statsig handles this cleanly.

From our portfolio observations: a well-run program brings a dormant customer back at twenty to thirty percent of the cost of acquiring a new one, measured as marketing spend per reactivation versus marketing spend per new acquisition over the same window. The secondary metric is cost per reactivation in absolute terms. The tertiary metric is false-positive rate: messages sent to customers who would have returned anyway. The Channel Selector's job is to keep that rate below ten percent (measured by holding out a control group from each cohort and comparing return rate).

Owner

Head of CRM or head of lifecycle marketing. The technical owner configures the trigger and the cost matrix. The lifecycle lead owns the message library, the brand voice rules, and the LTV cap policy.

What good looks like at L4

- The lifecycle lead has shipped two or three new subagents. Examples: a win-back-after-cancellation agent, a near-churn predictor that intervenes before the customer goes fully dormant, a lapsed-buyer agent that surfaces new products in their historical category.
- Reactivation rate of the dormant cohort versus a no-intervention control group: a meaningful lift at the 7, 14, and 30-day measurement points. Statistical significance computed weekly.
- Cost per reactivation trending down across iteration cycles, with channel mix shifting toward lower-cost channels as the agent learns each customer's response pattern.
- False-positive rate (messages to customers who would have returned anyway) below 10 percent. Measured by holding out a control group from each cohort.
- The lifecycle lead modifies the channel cost matrix and the LTV cap policy directly, without involving the technical owner.

Recipe 6 · Per-customer agents

Maturity level: L4. The hardest recipe in this part. The first recipe that requires the institutional memory layer from Chapter 9 to be in place.

Problem

Every consumer brand says they want "segment of one" personalisation. Almost none achieve it. The data is fragmented across email, push, WhatsApp, support, and on-app behaviour. Messages come from humans or templates that cannot process all that context. The result is segments of fifty thousand pretending to be segments of one.

Architecture

Swarm pattern. Each top-LTV customer gets a dedicated AI agent. The agent has one customer's history, one customer's preferences, one customer's lifecycle state. Does not context-switch. Writes all outbound messages, picks the channel, picks the time, picks the content. When the customer takes an action (or does not), the agent updates its plan.

This is the most ambitious recipe in the playbook. Start narrow. Top ten percent of customers by LTV. Ten thousand users is enough to prove the model. The institutional memory layer from Chapter 9 is mandatory: each agent reads and writes there, and that is how the system stays coherent without coordinator overhead.

Vector embedding architecture

Each customer gets a vector profile of roughly 150 dimensions across four spaces. Content affinity (what they engage with). Behavioural (when and how often they open, click, buy). Lifecycle (where they are in the customer journey, last action timestamps, expected next action). Engagement (channel preference, language preference, response pattern by day-of-week and hour-of-day).

Stored in a vector database. Pinecone, Weaviate, or Postgres with pgvector. The choice depends on scale and cost; Postgres pgvector handles up to a few million customers comfortably and is far cheaper than the managed alternatives below that scale.

Every customer interaction updates the vector. The per-customer agent reads the vector before composing any outbound message. New behaviour re-embeds the profile within an hour. Stale profiles drive bad decisions; freshness is the most important property of the vector layer.

This requires an event-driven pipeline: a webhook on every customer action, a queue (SQS or equivalent), a dedicated embedding worker. A nightly batch will not meet the freshness requirement.

Eval framework

Revenue per customer on the agent-covered cohort versus a segment-based control group. From our portfolio observations: 15 to 40 percent lift, measured over a full retention cycle (matched control cohort, full retention cycle; see “What good looks like” for setup). Secondary: reduction in wasted outreach spend, which can reach 70 to 90 percent at scale because the agent stops sending messages the customer would not engage with (measured as the share of outbound messages that received no engagement, comparing the agent-covered cohort against the segment-based baseline).

The eval is harder than for Recipes 4 and 5 because each customer is a sample size of one. The right unit of analysis is the cohort, matched on entry LTV and tenure, comparing the agent-covered group against a control group held out for the duration of the test. Run the test for at least one full retention cycle (typically 90 days for consumer brands, longer for subscription products with longer renewal windows).

Owner

Head of product or head of lifecycle marketing. Needs founder-level air cover throughout the pilot. The eval framework will produce mixed signals early. Some customers will respond to higher contact frequency; others will churn from it. The agent learns this. The founder has to give the program time.

What good looks like at L4

- Revenue per customer on the agent-covered cohort: 15 to 40 percent lift versus a matched control group. Measured over a full retention cycle.
- Wasted outreach spend: down 70 to 90 percent on the agent-covered cohort. Measured as the share of outbound messages with zero engagement.
- Top 10 percent of customers by LTV all have dedicated agents in production. Quantitative threshold: at least 10,000 active per-customer agents by end of pilot, scaling from there.
- The product or lifecycle lead modifies subagent behaviour directly. Each modification ships through the eval factory without involving the technical owner.
- Voice channel integrated with the per-customer agent. When the channel cost matrix calls for voice, the per-customer agent triggers the call. The voice transcript writes back to the institutional memory layer.

AN EXAMPLE FROM ONE OF OUR PORTFOLIO COMPANIES

At an India-native OTT platform in our portfolio, this recipe is in pilot at ten thousand users. The brand-character chat agents (separate from the per-customer agents) generate millions of conversations a week and have lifted subscription conversion by roughly one percentage point on exposed users (measured as the difference in paid-conversion rate between exposed and unexposed cohorts in the same acquisition window). Both patterns together (per-customer agents for top LTV plus character agents for engagement) are what L4 personalisation actually looks like at scale.

BORROWED INTELLIGENCE · THE PER-CUSTOMER PATTERN IS PORTABLE

One of our D2C beauty portcos has every ingredient for per-customer agents but no infrastructure to run them. Their creator-evaluation tool already processes more than a hundred data points per video. The same vector approach applied to customers would surface which products to recommend to which user. A pet healthcare portco we work with has dozens of clinics of consult audio sitting unmined. Per-pet agents (not per-customer, per-pet) running on transcribed consult history is the same recipe applied to a different domain.

Recipe 7 · Synthetic persona testing

Maturity level: L3 to L4. The eval surface that makes daily iteration safe. Closely connected to Chapter 3 (eval factory) but operates on a different time scale.

Problem

The team wants to iterate on outreach scripts, chat agents, and support bots daily. Every iteration on real customers is a production experiment that affects someone's experience. The team moves slowly because breaking real customers is scary. The eval factory from Chapter 3 catches regression, but it does not simulate full conversations. For that, the team needs personas.

Architecture

A library of 15 to 20 synthetic customers, generated by Claude from real conversation data. The technical owner pulls 12 months of real conversations. Claude clusters them by behaviour pattern. The responsive customer. The sceptical customer. The confused customer. The refund-seeker. The price-sensitive customer. The loyal customer. The churn-risk customer. Each cluster becomes a synthetic persona.

Each persona has three artefacts:

- A profile (demographics, motivations, common objections, vocabulary)
- An objection pattern (the typical sequence of responses the persona produces under different agent strategies)
- Twenty or more example messages from the source conversations, used to keep the persona's voice consistent

How the personas get used

Every new agent prompt runs through the persona-test skill before going live. The skill iterates through the personas, simulates a conversation between the agent and the persona, scores the outcome on a rubric, and reports back. Threshold: top 80 percent of personas rate the outcome "acceptable." Below that, the prompt does not ship.

The persona library also runs nightly against production agents on a sample of personas, catching drift. If the support agent passed the persona test at deployment but is failing the "sceptical customer" persona two months in, the agent has drifted and needs a refresh.

Eval framework

The primary metric is iteration velocity: the number of new agent variants the team ships per month. Teams typically move from weekly to daily after this is in place. The secondary metric is false-positive rate: the rate at which variants pass the persona test but underperform in production. Target: below 10 percent. If the false-positive rate is above 20 percent, the personas are not representative and need to be regenerated from fresh data.

Owner

Technical owner builds the library and the persona-test skill. Head of CX or head of brand owns the persona definitions for their function's agents. Personas are not static. They get regenerated quarterly from fresh conversation data, or when the false-positive rate (variants that pass persona testing but underperform in production) exceeds 20 percent on two consecutive monthly measurement cycles.

> **THE TASTE-TESTER PATTERN, MADE CONCRETE**
>
> The AI-native B2B SaaS company referenced in Chapter 3 built backtesting against historical data, simulated user behaviour, and large-scale A/B tests before their agent ever touched a customer. The personas in this recipe are the BVP-portfolio version of that machine. Build them once. Run them on every prompt change. The team will ship four times faster within two iteration cycles.

Recipe 8 · Overnight self-improvement loops

Maturity level: L4. The Karpathy autoresearch pattern. The recipe that compounds while everyone is asleep.

Problem

Agents are as good as the last human who reviewed them. Most companies ship agents in month three and look up at month eighteen with the agents still running the month-three prompts. The marketing has moved on. The customer behaviour has shifted. The competitive set has changed. The agent has not noticed any of it.

Architecture

A Claude Code skill called overnight-research, scheduled to run on a regular cadence (the portco decides what cadence works for them). Same pattern as Andrej Karpathy's open-source autoresearch framework, implemented natively in Claude Code with the Agent SDK. Four ingredients.

1. A target file the subagent can modify. The system prompt for a specific agent, a scoring function, a routing rule. One target per run.
2. A fixed time budget per experiment, typically five minutes. The subagent does as many variations as it can fit.
3. A quality metric that can be evaluated automatically. Same infrastructure as Chapter 3. The eval factory and the self-improvement loop share inputs.
4. A context file that gives the subagent direction: what the target does, what good outcomes look like, what failure modes to avoid.

Point it at the support agent’s system prompt and leave. Over the run window the loop tries 50 to 200 variants, measures each against the metric, keeps improvements, discards failures. Budget this from the API line, not per-seat allocation. Size per-agent API caps to cover overnight batch volumes before the first run. After the run completes the technical owner gets a Notion page with the change log, the current best, the top five variants, and a full rejection log of what did not work and why. Use Plan Mode in Claude Code to keep the loop’s actions visible and reviewable. Anthropic’s Founder Playbook makes this point directly: agents that can be inspected mid-run are agents that can be trusted.

The safety rules that keep this from going wrong

This is the recipe where things go most spectacularly wrong if the guardrails are absent. Three non-negotiable rules.

- The loop never deploys the winning variant automatically. Every promotion goes through a human approval. The Monday-morning Notion page includes a "promote" button. A human clicks it. The change ships through the same eval factory as any other prompt change (common, edge, adversarial, regression bar, shadow mode).
- The loop never modifies its own context file or its own time budget. Those are owned by the technical owner. An agent that can rewrite the rules of its own improvement loop is an agent that will find a way to game the metric.
- The loop logs every variant it tried, the score it received, and the reasoning the judge gave. Stored permanently. Read-only after the run completes. If something goes wrong in production later, the trail is intact.

Eval framework

Per target: percentage improvement in the metric over a full cycle of the loop, net of human review time. From our portfolio observations: 10 to 30 percent improvement in the first cycle is typical. Compounding returns when 5 or more loops run in parallel on different targets. The portco defines the cycle length; week-on-week improvement is noisy, so longer cycles produce cleaner signal.

Secondary metric: rejection-log learnings. The variants the loop tried and the judge rejected are data. Patterns in those rejections inform the technical owner about what the agent thinks the metric is asking for, which is often subtly different from what the founder thinks the metric is asking for. That gap is where future improvements live.

Owner

Technical owner only. The one recipe in this book where the founder stays out of the weeds. Self-improvement loops are operational infrastructure that one person owns and ten people benefit from.

What good looks like at L4

Five to ten loops running in parallel, each targeting a different agent or scoring rule. Notion pages with concrete changes the technical owner promotes through the eval factory. From our portfolio observations: cumulative improvement of 30 to 60 percent on the targeted metrics over the first year, measured against the pre-loop baseline. At an India-native OTT platform in our portfolio, the brand-character chat agent's inappropriate-message rate dropped roughly 70 percent after an auto-improvement loop began rewriting

the system prompt (measured as the share of agent responses flagged for content-policy review). The same pattern applies to every agent in this playbook.

Beyond prompts: agents that improve their own tools

Recipe 8 as described above is an agent that rewrites its own prompt. The next level is an agent that rewrites its own tools. This is the frontier and the most powerful version of the self-improvement loop.

All safety rules from the prompt self-improvement section apply to code changes. Additionally: every loop-proposed code change must pass the Recipe 10 PR reviewer eval gate before human review. Maximum diff size for a loop-proposed change is 100 lines.

The pattern: a base agent answers queries using a set of deterministic tools (database reads, calendar lookups, RAG over a corpus, custom code). A second agent monitors every query the base agent runs, tracks which succeeded and which failed, and asks for each failure: what would have made this query work? Did we need a different tool? A different database view? A new index? A better skill?

When the monitoring agent identifies a fix, it writes the code, submits a pull request to the repository, gets the PR reviewed by another agent, merges it, and deploys. The next time a human asks the same query that previously failed, it succeeds. The system is improving the infrastructure that the base agent runs on, not just the prompt.

This is the architecture that makes a company actually self-improving while the team is asleep. The safety rules from earlier in this chapter all still apply: no autonomous promotion, the eval factory gates every merge, the audit log captures everything. The difference is what the agent is allowed to change. Prompts are the safer target. Tools and code are the higher-impact target.

> **WHEN TO EXTEND RECIPE 8 TO TOOL SELF-IMPROVEMENT**
>
> Run prompt-only self-improvement for at least one full eval cycle before extending the loop to tools and code. The portco needs the muscle memory of reviewing prompt rewrites before it takes on the harder review surface of code rewrites. Once the technical owner is comfortable with the prompt loop, the same architecture with a wider write scope is the obvious next step. Anthropic's Founder Playbook calls this the moment Claude Code transitions from engineering assistant to part of the operations layer. It is the inflection from L3-and-a-half to real L4.

> **BORROWED INTELLIGENCE · KARPATHY'S PATTERN, ADAPTED**
>
> Andrej Karpathy released autoresearch as an open-source reference for this exact pattern. The engineering team at an India-native OTT platform in our portfolio studied it, picked up the four-ingredient architecture, and re-implemented it in Claude Code with the Anthropic

> Agent SDK. The reference repo is the right starting point. Copy the architecture, not the code; the Claude Agent SDK does most of the heavy lifting Karpathy's repo had to do manually.

Recipe 9 · Institutional memory as a queryable layer

Maturity level: L3 to L4. The Garry Tan GBrain pattern. The infrastructure every other agent in this part queries on every run.

Problem

Every company accumulates knowledge: customer research, campaign reports, competitor intelligence, SOPs, founder memos, meeting notes. Most of it rots in Notion, Drive, and someone's Obsidian vault. New hires spend three months catching up. Agents miss context that exists somewhere but is not accessible.

Garry Tan's open-source GBrain solves this for individuals. The institutional version, implemented inside the company's existing tooling, is what powers L4.

Architecture

Every entity that matters gets a record in the institutional memory layer. Customer. Competitor. Product SKU. Campaign. Vendor. Investor. Each record has two parts.

- A "compiled current truth" at the top of the record. Two to four sentences. The current consensus view, rewritten by an agent whenever significant new evidence arrives. This is what other agents read first when they pull the entity for context.
- An append-only evidence timeline below it. Every email, meeting note, support ticket, press release, internal memo, or customer interaction that touches the entity gets a row in the timeline. Source, date, summary, link. Never edited. Never deleted.

How professional intelligence analysts maintain assessments. The compiled current truth is the analyst's working hypothesis. The evidence timeline is the case file behind it. Garry Tan's GBrain implements this exactly. Replicate it in Notion (databases with two block types per entity), in Postgres (two tables with a foreign key relationship), or in a hybrid (Notion for the compiled truth that humans need to read, Postgres for the timeline at scale). Claude Code maintains both on a heartbeat.

What you actually store

Treat the institutional memory layer as a working operational system, not a permanent archive. Three rules govern what goes in and how long it stays.

- Store the minimum personal data the agents need to act. If the agent does not use a field to make a decision, do not store the field on the entity record. Logs and source artifacts can live in their own systems with their own retention; the memory layer holds only what feeds the loop.
- Attach simple retention defaults per entity type. The technical owner can adapt these to the company's specific situation; they are defaults, not legal rules.
- Build a redact-on-request path that removes personal fields from a customer or employee record on request, while preserving the non-personal statistics the loops still need. A redacted entity is marked as such; the timeline retains its operational summaries but personal identifiers are removed.

Default retention rules the technical owner can adapt:

ENTITY TYPE	DEFAULT RETENTION
Customer interactions	While the customer is active, plus one renewal cycle after churn.
Sales call transcripts	While the deal is open, plus the full contract term if the deal closes.
Internal meeting transcripts	Duration of the related workstream plus one quarter.
Competitive intelligence	Indefinite. Competitor records compound in value over time.
SKU and campaign performance	Indefinite. Historical performance data feeds future planning.
Employee 1:1 notes and performance documentation	Governed by the company's HR policy, not by the memory layer's defaults.
Vendor and contract records	Active contract term plus the longer of two cycles or any audit window the contracts specify.

These defaults exist so the technical owner has something to start with rather than a blank policy document. The company's legal and compliance team writes the actual policy. The technical owner wires it into the heartbeat process and the redact-on-request path. Those two together are the operational implementation of whatever the policy says.

How agents query the layer

Every agent in the playbook gets an MCP connector to the memory layer. When the marketing agent pulls a customer for an outbound campaign, it reads the compiled current truth (current engagement status, last action, known preferences) and queries the timeline only if the truth is more than a week old or seems incomplete.

Reading the compiled truth is cheap. Reading the full timeline is expensive. The two-tier design is what makes the layer scalable to millions of entities without every agent call hitting the full corpus.

How the layer updates itself

Claude Code runs a heartbeat process every fifteen minutes. It scans the recent inputs (new emails in Outlook, new tickets in Zendesk, new entries in the CRM, new meeting notes in Notion) and identifies which entities they touch. For each touched entity, it appends a timeline row. If the new evidence changes the compiled current truth substantially, the agent rewrites the truth and flags the rewrite for human review.

The heartbeat must fail loudly. Write a status row on every run. Alert when two consecutive heartbeats fail or entities-updated drops to zero.

Substantial means: the agent's confidence in the new version of the truth exceeds the previous version by more than a defined threshold, or the rewrite contradicts a statement in the previous version. Both trigger human review. Minor updates (a new evidence row with no implication for the truth) do not trigger review.

Eval framework

Two metrics. The primary one: percentage of agent queries to the layer that return useful context. "Useful" is judged by the agent itself (does the returned context change the agent's next action) and verified weekly by sampling. Target: above 70 percent within the first iteration cycles. Above 85 percent as the layer matures.

The secondary metric: new-hire ramp time. The qualitative test. A new marketing hire on their second or third day can answer a question about an old campaign by querying the Marketing Project, which queries the memory layer. The layer returns the compiled truth, the five most relevant timeline entries, and the links. New-hire ramp time drops by an order of magnitude in companies where this works.

Owner

Technical owner builds and maintains the heartbeat. The chief of staff or operations lead owns the editorial standards (what entities matter, what the compiled truth should look like, what triggers a rewrite review). Both partners must be in place. A technical owner without an editorial partner produces a memory layer that is technically correct and operationally useless.

Make the organisation legible to AI

The institutional memory layer only works if the inputs that feed it actually exist. The default state of most companies is that the inputs do not. Conversations happen and disappear. Meetings happen and only the rough notes survive. DMs scroll past. The brain of the company is in fifteen heads, half of which will move on at some point.

The shift is to record everything that can be recorded. Partner emails. Slack messages and DMs. Internal meetings. Customer calls. Sales calls. If it happened and was not recorded, it did not happen to the AI. If a conversation matters enough to act on, it matters enough to record.

Recording produces volume the AI cannot consume directly. Hundreds of hours of audio do not fit in any context window. The heartbeat process from earlier in this chapter is what diarises the audio (separates speakers), summarises it, and writes the relevant fragments to the entity timelines. The compiled current truth at the top of each entity record is the synthesis. The raw recording is preserved for audit but rarely read directly.

A useful test: a new hire on their second day can ask the Marketing Project a question about a fourteen-month-old campaign and get back the compiled current truth, the five most relevant timeline entries, and links to the underlying recordings or emails. The new hire's ramp time drops by an order of magnitude. The institution is now legible to anyone (or any agent) who needs to read it.

> **THE SELF-REGENERATING USER MANUAL**
>
> Take the team’s recorded internal sessions (training, office hours, all-hands), diarise and categorise the audio, then use the corpus to regenerate the user manual or operating playbook the team relies on. The regenerated document is materially better than the version written years earlier, produced in a single iteration. The same pattern applies to every operating document in a portco: the brand book, the sales playbook, the onboarding guide, the engineering runbook. Treat the document as ephemeral. Treat the raw inputs and the synthesis skill as the asset. Regenerate the document whenever the underlying inputs have shifted.

Ephemeral software, preserved data

Related point that matters at L4. Most internal software (dashboards, ops tooling, function-specific workflows) is now cheap enough to regenerate. Claude Code one-shots most internal dashboards to a high quality. The valuable asset is not the code; the valuable asset is the understanding inside the team's heads about what the dashboard should show and why.

Treat internal software as ephemeral. Treat the original specifications, the design intent, and the underlying data as preserved. When models get smarter, regenerate the software. The understanding that produced the original specification is what survives. The implementation is replaceable. Anthropic's Founder Playbook makes the same point: in an AI-native company, Claude Code becomes part of the operations layer, and the company stops treating internal-tools code as an asset to defend.

Recipe 10 · The engineering orchestrator

Maturity level: L3 to L4. The flagship engineering-native recipe. A coordinator pattern that spans the entire software-shipping pipeline from PRD to production. The most impactful recipe in this part for B2B SaaS portcos and any engineering-led consumer portco.

Problem

The standard engineering workflow has a known shape. A product manager writes a PRD. Someone breaks it into Jira tickets. Engineers pick up tickets, write code, open pull requests. A reviewer reads the diff and asks for changes. CI runs tests. A lead approves. The PR merges. A release manager bundles changes for deployment. Each handoff is a human bottleneck. Half of those handoffs are waiting for a human who is in a meeting, on a different timezone, or working on something else.

At L4, most of the handoff work is agent work. Humans review at defined checkpoints. The lead time from PRD approval to merged PR collapses by a multiple. Engineering capacity stops being measured in headcount and starts being measured in concurrent agent runs.

Architecture

Coordinator pattern with five subagents. Build it inside Claude Code as a top-level skill called engineering-orchestrator. The Engineering Project holds the coding standards, the architectural decisions, the test strategy, and the CLAUDE.md file at the root of every repo. Every subagent reads from this Project before acting.

SUBAGENT	TRIGGER	WHAT IT DOES
PRD-to-tickets	On PRD approval in Notion or Linear	Reads the PRD, decomposes into Jira tickets with acceptance criteria, dependencies, and estimated complexity. Posts the proposed ticket set to a Slack channel for human approval before creating.
Coding	On Jira ticket assigned to the agent (specific label)	Reads the ticket, the related code, the test strategy. Writes or edits the code. Opens a draft PR with the diff, a description, and the tests it added.

Test runner	On new PR opened by the coding agent	Runs the full test suite plus any tests the coding agent added. Posts pass/fail back to the PR with a structured summary. Re-runs after every push.
PR reviewer	On PR ready for review (label)	Reviews the diff for correctness, style, architectural fit, and risk. Posts inline comments. Blocks merge if catastrophic issues found. Flags the human reviewer for anything ambiguous.
Release manager	Daily, or on demand	Aggregates pass/fail status across recently merged PRs. Proposes deploy bundles. Posts the bundle to Slack for human approval before deploying.

The Release manager is optional. Most portcos can ship Recipe 10 without it and let humans handle deploy bundling. Add the Release manager when deploy cadence exceeds a few per day and the bundling itself becomes the bottleneck.

Subagent measurable outputs

- PRD-to-tickets: number of tickets proposed per PRD, percentage of tickets approved without human edit, dependency cycles caught before ticket creation.
- Coding: number of PRs opened per cycle, percentage of agent PRs passing CI on first push, percentage of agent PRs requiring no human commits before merge.
- Test runner: time from PR open to first CI signal, percentage of test runs that complete without infrastructure errors.
- PR reviewer: percentage of human-flagged issues the reviewer caught independently, false-positive rate on style and risk flags, regression rate when the reviewer's prompt changes.
- Release manager: percentage of proposed bundles approved without human edit, time from PR merge to deploy.

The famous persona trick, applied here

Same pattern as Recipe 4. PRD-to-tickets is Leslie Knope from Parks and Recreation (thorough, organised, leaves nothing implicit). The coding agent is John Carmack (real engineer, prefers correctness and minimal abstraction). The test runner is Columbo (always finds the small thing everyone else missed). The PR reviewer is Atticus Finch (fair, thorough, defends quality while respecting the engineer). The release manager is Dwight Eisenhower (operations, coordination, says no when things are not ready).

How to wire it

- Connect Claude Code to the GitHub repo via MCP. Read access to all repos, write access only to a sandbox repo until the eval factory is calibrated.
- Connect Claude to Jira via MCP. Read all tickets, write only to a designated agent-managed swimlane.
- Build the Engineering Project in Claude. Commit coding standards, architectural decisions, the test strategy, and the team's PR templates. Every subagent reads this on every run.
- Write the CLAUDE.md file at the root of every repo. The coding agent reads this first before any edit. It defines what to follow, what to avoid, and the conventions specific to that codebase.
- Build the engineering-orchestrator skill. Top-level skill that calls each subagent in the right order with the right context.
- Use GitHub labels and Jira labels to gate which tickets and PRs the agents touch. A ticket labelled agent-eligible gets routed to the coding subagent. A ticket labelled human-only stays out of scope.
- Start in draft-only mode. Agents open draft PRs and post test summaries. Humans approve every merge to main. Move to direct-merge on agent-managed PRs only after the eval gates below have stabilised.
- Scope the coding agent explicitly in CLAUDE.md: which directories it can touch, which dependencies require human review, maximum diff size of 200 lines per PR.
- Move out of draft-only when CI pass rate is at or above 85 percent on 50 consecutive agent PRs, PR reviewer catch rate is at or above 70 percent on the golden eval set, and the engineering lead has reviewed 20 agent PRs and is satisfied.
- Wire CI/CD outputs back into the orchestrator. Test results, lint results, and security scans all flow back to the subagents so the coding agent learns what passes and what does not.

Eval discipline: a hybrid

Code is harder to evaluate than ticket text or call transcripts. The honest answer is two eval surfaces, one each for the two distinct quality questions.

Primary surface: standard CI/CD. The same tests that gate human PRs gate agent PRs. No exception. The coding agent's CI pass rate becomes the primary quality metric. If a human PR would have failed CI for a given change, the agent's PR fails too. This is the floor.

Secondary surface: a code-review eval. Build a golden set of 20 to 50 historical PRs where the team knows what good review looked like. For each PR in the set, record what the human reviewer caught (real bugs, style issues, architectural problems) and what they appropriately let through. The PR reviewer agent runs against this set on every prompt change. Score on three rubrics.

- Catch rate: percentage of real issues from the gold-standard review that the agent identified.
- False-positive rate: percentage of agent flags that the gold-standard review explicitly let through.
- Style adherence: agent comments match the team's review tone and standards rather than introducing arbitrary preferences.

Regression bar: when the PR reviewer prompt changes, no drop more than three percentage points on catch rate, no increase more than three points on false-positive rate. Promotion gate follows the same shadow mode discipline as Chapter 3. The new prompt runs in shadow for a defined window where it posts comments to a hidden channel; the technical owner samples and approves before promotion.

> **WHY TWO SURFACES AND NOT ONE**
>
> CI/CD tells the technical owner whether the code works. The code-review eval tells the technical owner whether the agent is reviewing the way the team would. Both questions matter at L4 because the agent is doing the work a senior engineer used to do, and senior engineering review is the layer that catches the issues CI cannot.

Owner

Engineering lead or CTO owns the recipe outright. The technical owner builds the orchestrator skill and configures the subagents. The engineering lead owns the labels, the review standards, the eval golden set, and the promotion gate. The handoff back to engineering ownership is what makes this an L4 capability rather than a vendor-built tool.

What good looks like at L4

- Lead time from PRD approval to merged PR for agent-managed tickets: a portfolio observation in the range of 50 to 70 percent faster than the manual baseline. Measured as wall-clock from Jira ticket creation to PR merge, for tickets labelled agent-eligible, against the team's previous 90-day baseline on comparable tickets.

- Percentage of PRs fully drafted by agents: above 60 percent of the team's monthly PR volume. Measured as PRs where the first human commit on the branch happens after the agent has opened the PR and posted the first set of test results.
- Test failure rate on merged agent-written PRs versus manual baseline: within 2 percentage points. Measured as post-merge revert rate plus post-merge hotfix rate, computed on a 30-day rolling window.
- Engineering time on code review per week: a portfolio observation of 40 to 60 percent reduction. Measured as time logged against PR review activity, captured via GitHub event stream and Jira time tracking.
- New-engineer ramp: a junior engineer ships their first agent-assisted PR on day one. Qualitative test; either it happens or it does not.

BURN TOKENS, NOT HEADCOUNT · THE ENGINEERING CASE

Recipe 10 is the recipe where the burn-tokens math becomes the most obvious. A senior engineer costs the company more per cycle than the entire monthly token budget for the engineering orchestrator running at scale. The unit decision is no longer "do we hire a third engineer." It is "do we widen the agent's scope to absorb the work a third engineer would have done." Most engineering-led portcos at L4 will run hot on tokens before they hit a true headcount constraint. That is the right shape.

WHERE RECIPE 8 MEETS RECIPE 10

Once Recipe 10 is stable and the golden PR eval set has 50 or more examples, point Recipe 8’s overnight loop at the PR reviewer’s system prompt. The catch rate is the clearest improvement signal in the entire stack.

Voice-native operations

Production-grade voice. The Part I voice recipe got the company to a working voice agent. This chapter is about voice as an operating layer.

Before any of the architectural detail below, one operating principle: record and transcribe as many calls as you can justify operationally. The more call context the company's agents see, the better they get at routing, summarising, drafting, and personalising. Wire the consent and data-handling practices your legal and compliance team are comfortable with into the stack from day one, and let those constraints shape the implementation. The legal framing is theirs to own. The operational principle (capture context broadly, route it into the memory layer from Chapter 9, retain on the defaults that chapter defines) is what makes the rest of this chapter compound.

Why voice deserves its own chapter at L4

Voice is the next major shift in AI adoption. India specifically has both the customer-facing demand (Tier 2 and Tier 3 cities where phone is the dominant channel) and the supply-side advantage (Sarvam AI's Indic-language voice models). For consumer portcos, voice is the biggest single unlock between now and end of 2027. For SaaS portcos selling into call-centre-heavy industries, voice is the agent product.

The production stack

The components and their roles. Pick this stack unless a portco has a specific reason to deviate.

LAYER	TOOL	WHY
Reasoning	Claude (via API)	The brain. Decides what to say next.
Speech to text (English)	Deepgram Nova-3	Best accuracy on Indian English and Hinglish code-switching.
Speech to text (Indic)	Sarvam AI	Outperforms global models on Hindi, Tamil, Telugu, Kannada, Marathi, Bengali, Gujarati. Built for Indian phone-quality audio.
Text to speech (English)	ElevenLabs	Latency under 300ms. Natural cadence. Voice cloning if brand requires.
Text to speech (Indic)	Sarvam AI	End-to-end Indic. Sub-second latency on streaming.

Telephony	Exotel	Indian regulatory compliance built in. Connects to existing call infrastructure.

Streaming is non-negotiable

Audio streams phoneme by phoneme, not as a completed file. Without streaming, every AI voice sounds like Siri from 2014 and customers hang up. The first 500 milliseconds of a response are what the customer judges the agent on. Latency targets:

- First syllable: under 500ms from the end of customer speech
- Full first sentence: under 1.5 seconds
- Maximum acceptable gap mid-sentence: under 300ms

Sarvam handles Indic streaming natively. Deepgram and ElevenLabs handle English natively. If the stack adds Latency, it almost always traces to the orchestration code rather than the individual components. Profile aggressively.

Use cases

Three use cases dominate. Inbound support: a voice agent answers the call, handles the common cases, escalates to a human when out of scope. Outbound retention: dormant reactivation, trial nudges, post-purchase follow-up. Sales qualification: the first call of the sales process replaced by an AI that books a meeting with a human only when the lead is qualified.

One of our travel portfolio companies piloted outbound voice for the first sales call and saw 25 to 28 percent pickup rates in initial testing, though the CTO acknowledged response latency was creating obvious AI detection. The latency targets above are what fixes that. Pre-optimisation figure, captured before latency was resolved. Treat as a floor.

Eval framework

Three metrics. Pickup-to-resolution rate for inbound: of calls that get answered, what percentage get resolved without human escalation. Target: 80 percent. Customer-perceived naturalness, measured by post-call survey on a 1-to-5 scale. Target: 4 or above. Cost per resolved call. Target: 10 to 20 times lower than a human call centre.

Owner

Head of CX for inbound. Head of lifecycle marketing for outbound. The technical owner sets up the orchestration, the latency budgets, and the streaming layer. The function lead owns the conversational design, the brand voice, and the escalation logic.

What good looks like at L4

Voice is no longer a separate product. It is one of the channels the per-customer agent from Recipe 6 picks when the channel cost matrix calls for it. The agent decides, based on the customer's history, that this particular interaction warrants a voice call rather than a WhatsApp. The voice agent handles the call. The transcript writes back to the institutional memory layer from Recipe 9. Every voice interaction makes the next text interaction smarter. Channels stop being silos.

Long-horizon agents and computer use

Maturity level: L4. The frontier. Agents that work for hours or days unsupervised, including on systems with no API.

The shift this chapter covers

Most of the recipes in this book assume agents that complete a task in seconds and return a result. The agents in this chapter run for hours, sometimes days. They take checkpoints. They wait for human input at defined moments. They handle systems that have no API by driving the browser the way a human would. This is where the autonomy slider Karpathy described in his 2025 talk gets pushed further than most of the recipes above need it to go.

Anthropic Computer Use

The Computer Use API gives Claude the ability to take screenshots of a virtual machine, interpret them, and issue keyboard and mouse commands. This is the technical primitive behind every browser-driving agent in the BVP portfolio at L4.

Use computer use when an essential business system has no API. Indian airline booking portals. Government compliance portals (GST, EPF). Quick-commerce supplier dashboards that have not opened developer endpoints. Old-school B2B portals that require login and manual data entry. One of our travel portfolio companies has fifteen-plus B2B and B2C booking portals that fit this profile exactly. The CTO knows the ops layer is fully manual and considers it the next frontier. Computer use is the recipe.

Multi-day workflows

When a workflow spans days (a customer who has not responded in three days needs follow-up; a campaign that needs daily monitoring for two weeks; a sales sequence that runs across five touchpoints over a fortnight), the orchestration layer matters more than the agent itself. Three patterns work.

- Cron-scheduled with state in a database. The agent reads its state from Postgres on every run, decides the next action, writes the new state, exits. Simple, scales well, costs almost nothing to run.
- Temporal or Airflow workflows. When the steps have complex dependencies, retry logic, and need durable execution across infrastructure failures. The Anthropic Computer Use examples ship with Temporal integration.

- n8n for the cases between simple cron and full Temporal. Visual workflow builder. Connectors to most SaaS tools. Reasonable cost. Useful when a non-engineer needs to be able to modify the workflow.

The supervision model

Every long-horizon workflow has defined human checkpoints. The technical owner names these explicitly when the workflow is designed. Three categories of checkpoint.

- Approval checkpoints. The workflow pauses and waits for a human approval before taking an action. Used for any Tier 3 action (irreversible, financial, external-facing) from the Part I action-irreversibility ladder.
- Review checkpoints. The workflow completes a phase and posts to a Slack channel with a summary. The human reviews asynchronously. The workflow continues unless the human intervenes within a defined window.
- Escalation checkpoints. The workflow encounters an unexpected state and halts. The technical owner gets paged. The workflow does not resume until a human acknowledges the situation.
- Hard session time-outs. Set one for every Computer Use task. 30 minutes is a reasonable default. When the limit fires, the agent surfaces a failure summary and queues for human review.

> **WHEN NOT TO USE A LONG-HORIZON AGENT**
>
> If the task can be done by a single short agent call followed by a queue and a scheduled follow-up, do that instead. Long-horizon agents are operationally expensive: more state to manage, more failure modes to handle, more observability infrastructure required. The right choice is the simplest architecture that solves the problem. Reach for computer use and multi-day orchestration only when nothing else works.

Eval framework

Two primary metrics. Completion rate: the percentage of workflow runs that finish all steps without an escalation checkpoint firing. Target: above 90 percent once the workflow has stabilised. Below that, the workflow design is wrong and needs to be decomposed into shorter chunks. Time-to-completion: the median wall-clock time from workflow start to workflow end. Track the distribution, not just the median; long-tail blowups are where costs accumulate.

The org chart of an AI-native company

L4 is not a tooling change. It is an organisational change. The recipes in this part assume a company that has restructured to support them. This chapter covers what that restructure looks like, drawn from the publicly documented transition of an AI-native B2B SaaS company in our portfolio and validated against what is working inside the BVP India portfolio.

Burn tokens, not headcount

The starting point. The unit economic shift that makes everything else in this chapter possible. The publicly visible early-stage AI-native cohort is reaching first-growth-stage milestones with materially higher revenue per employee than companies of comparable age eighteen months earlier. The trend continues through Series A and Series B.

For a BVP portco, the implication is concrete. The next hire decision is not "do we hire an analyst, an SDR, or a coordinator." It is "can a token spend at this scale replace this function entirely." The blunt directional measure: track per-employee token usage. It is gameable at the extreme and dumb to turn into a leaderboard. Directionally it tells the founder which employees are figuring out the new way of working and which are still on the old one.

The teams that get this right end up constrained on token spend rather than headcount. That is the L4 budget conversation. Founders should be running it explicitly with their CFO and their board.

Middle management is done

Most companies are still organised like Roman legions. Nested hierarchies, with humans acting as the conduit for information flowing up and down. The middle management layer exists primarily to route that information. AI does the routing now, faster and at lower cost.

Two roles matter for an L4 company. Individual contributors who build and operate (the product-minded generalist below is one version of this profile). And Directly Responsible Individuals (DRIs) who own outcomes. A DRI is a single named human, not a committee, responsible for a defined surface. Middle management as a coordination layer in between the two becomes redundant. The reporting structure flattens.

This is uncomfortable for portcos with existing middle layers. The honest read is that the transition takes a year or two and produces a leaner, faster org. The companies that complete the transition outperform the ones that postpone it.

Centralise R&D with single-threaded ownership

The AI-native B2B SaaS company referenced above made centralisation their first move. They grew the AI team from fewer than 10 to nearly 50 ML researchers and scientists. Did not disperse them across product teams. Measured them not by what they shipped each iteration but by what they learned. The choice protected the team from the pressure of constant shipping and let them pursue fundamental improvements rather than local optima.

For a BVP portco at Series A or B, the equivalent is one or two people centrally responsible for the AI infrastructure that everyone else uses. Not embedded in marketing, not embedded in CX, not embedded in engineering. Reporting directly to the founder or to the CTO. Their metric is the rate at which other functions ship their own agents, not the agents they ship themselves.

Inside the work itself, single-threaded ownership. The same company calls these workstreams: ten-to-fifteen person teams pulled from engineering, ML, sales, marketing, and design, each assigned a Directly Responsible Individual with the autonomy to drive a specific project. The DRI may be from any discipline. The DRI owns the outcome. Almost every R&D person at that company today works on the flagship AI agent in some form.

Re-architect the codebase for AI

The same publicly documented case study covers this. The company migrated its entire front end from one JavaScript framework to another. Not because the new framework was trendy. Because, as their SVP of Engineering put it, AI is meaningfully better at writing code in the new framework than the old. The codebase that AI cannot efficiently read, write, and refactor becomes a liability.

For the BVP portfolio, this implies three concrete things. First, audit the codebase. Identify languages, frameworks, and patterns where Claude struggles. Plan a migration if the volume of work justifies it. Second, write a CLAUDE.md file in the repo root that codifies the architectural decisions, the patterns the team follows, and the dependencies the team avoids. Every Claude Code session reads this file first. Third, deploy autonomous coding agents that proactively submit pull requests for low-risk maintenance: dead-code removal, dependency updates, test-coverage gaps. The pattern is portable.

The three hires the L4 company makes

Three profiles. Hire all three. Balance is the point.

PROFILE	WHAT THEY DO	WHERE THEY LIVE IN THE ORG

Product-minded generalist	Owns problems end-to-end. Builds. Talks to customers. Ships features. Comfortable with AI as a tool but not a researcher.	Inside every functional workstream.
Design engineer	Designs and ships production code. Eliminates handoffs between design and engineering. Common in AI-native companies; rare in traditional ones.	Embedded in product and creative teams.
Deep specialist	ML PhD or staff engineer with experience shipping production AI. Few in number. Concentrated at the centre.	In the centralised AI team. Two to four people for a Series A-B company. More at Series C and beyond.

FROM A PORTFOLIO COMPANY WE WORK WITH

AI workflow creation is a criterion in performance reviews for customer-facing roles at one of our SaaS portcos. The only structural adoption mechanism we saw in the BVP portfolio for non-engineering functions. The pattern: when an employee's review explicitly evaluates how many AI workflows they have built and how widely those workflows have been adopted by others, the adoption curve goes vertical. Without it, AI adoption stays in the hands of the few who would have adopted on their own anyway.

Humans on the edge, AI in the middle

The company brain (skills, knowhow, data, memory layer, the institutional context) sits in the middle. Humans live on the edge, where the intelligence makes contact with reality.

Humans reach into places the models cannot reach yet. Live customer meetings. Conference rooms. High-stakes ethical calls. The conversation where a founder is thinking about breaking up with a co-founder. Real high-emotion, high-stakes moments where the customer or partner wants a human in the room. Sales for the foreseeable future. Board conversations. Crisis moments. These stay human.

Everything else (information routing, document production, dashboard generation, data synthesis, scheduled outreach, recurring decisions) gets handled by the brain. The org chart stops looking like a pyramid and starts looking like a doughnut. Humans on the rim. AI in the centre.

Forward-deployed engineers for enterprise

The same case study introduced forward-deployed engineers who work on-site with customers to ensure successful deployment. The shift the SVP described: traditional

SaaS sales is like selling a car, a transactional exchange that leaves risk with the customer. AI sales is like teaching somebody to drive, a consultative partnership.

For BVP SaaS portcos selling AI agents into enterprise, forward-deployed engineering is the right GTM motion. The forward-deployed engineer sits inside the customer's relevant function for the first several weeks, configures the agents to their specific contracts and logic, and trains the customer to maintain the configuration. The contract structure that supports this is in Chapter 14.

What gets harder

Two things. Hiring becomes more selective. The product-minded generalist who builds and ships is harder to find than the engineer who codes against a spec or the marketer who executes a brief. Interview processes have to update. Take-home projects that involve building with Claude Code, design exercises that require shipping a working prototype: these matter more than algorithm questions.

And the culture shift. A company that promises every team member they will be a builder, and then makes them sit in approval queues, will lose its best people quickly. The permission model has to reflect the org chart. Tier 1 actions auto-approve. Tier 2 actions get fast human approval. Tier 3 actions get the right approval, no more. The Part I action-irreversibility ladder is the contract.

Pricing for AI-native B2B

This chapter applies primarily to the SaaS portcos in the BVP India portfolio. Consumer portcos can skip to Chapter 15. For a B2B SaaS portco selling billing or finance infrastructure, this chapter is the operating manual.

Why per-seat pricing breaks at L4

Traditional SaaS pricing assumes the customer is paying for human seats accessing the software. An AI-native product replaces the seats. The customer's three-person finance team becomes a one-person team plus an agent. The seat-count metric is now a measure of inefficiency the buyer is actively working to reduce.

Per-seat pricing in this context is a structural revenue risk. The buyer's incentive is to reduce seats. The vendor's incentive is to grow seats. Misalignment guaranteed. The 2026 VC Playbook (Faisal Mushtaq, March 2026) names this as a primary red flag in due diligence.

Outcome-based pricing

The AI-native B2B SaaS company referenced earlier pioneered the model: their flagship agent charges 99 cents per autonomously resolved customer conversation. If the agent escalates to a human, the customer does not pay. Pricing tracks the value delivered rather than the resource consumed. Salesforce and Zendesk announced similar models afterward. The shift is industry-wide.

For a portco selling billing infrastructure, the equivalent unit is the autonomously processed billing event. A complex invoice that the agent handles end-to-end without human intervention. An automated reconciliation. A finalised contract renewal where the agent extracted terms from the email thread and updated the system. Each one is an outcome the customer would have paid a human for. The pricing tracks that.

How to structure the contract

Three components. A base fee that covers the platform access (lower than the seat fee would have been). A per-outcome fee for each successful automated action. A volume tier that drops the per-outcome fee as usage scales. The buyer's incentive is to push more volume through the platform. The vendor's incentive is to handle more outcomes successfully. Aligned.

The contract also defines what counts as a successful outcome and who arbitrates disputes. Both sides need a clean definition. The eval factory from Chapter 3 is what produces the arbitration data: the audit log of every action, with the model version, the rubric score, and the customer's acceptance signal. The audit log is part of the contract.

Forward-deployed engineering as the GTM motion

Outcome-based pricing requires the customer to actually achieve outcomes. That means the vendor has to get the agent configured to the customer's specifics. For complex B2B products, that is a multi-week engagement. Forward-deployed engineers do this work.

The cost model: a forward-deployed engineer costs the vendor 5 to 10 thousand dollars per deployment cycle for an Indian customer, 15 to 25 thousand dollars per cycle for a US one. Engagement durations are measured in iterations rather than weeks; the portco decides what cadence works. The customer-success cost is real but the contract value is higher and the gross retention is far better. The publicly documented case study validated this. The math works for any AI-native B2B SaaS company selling into enterprise.

The sequence

The order in which to build the Part II stack, with no predefined timeline. Each portco moves at the pace they can sustain. Some will compress this into a single quarter. Others will spread it across two years. Both are valid. The sequence matters more than the schedule.

> **WHY NO TIMELINE**
>
> Every portco has different scale, different existing infrastructure, different appetite for change. Prescribing a calendar is wrong. What is right is the dependency order: which steps unblock the next ones. Run the steps in order. Run them at whatever pace the company can absorb.

Step 1: Install the eval factory

The first move. Before any new agent ships and before any existing agent gets a prompt change, build the eval factory from Chapter 3. Golden datasets for every Part I production agent. LLM-as-judge with rubrics. Promotion gate. Shadow mode. Continuous evals.

Nothing in Part II works without this. A portco that tries to ship multi-agent architectures or self-improvement loops without eval discipline produces fragile demos that fail in production. Step 1 is the gate.

Step 2: Ship the first coordinator pattern

Pick one. Recipe 4 (performance marketing) for consumer portcos with meaningful Meta or Google spend. Recipe 5 (dormant recovery) for portcos with large dormant cohorts. Recipe 9 (institutional memory) for portcos where context is the bottleneck.

Build the coordinator. Configure the subagents using the famous persona trick. Run in reporting-only mode until the override rate stabilises. Grant write access on the lowest-risk actions. The portco's first multi-agent system in production. This is where L3 becomes real.

Step 3: Build the persona library and the memory layer

Two parallel tracks. The persona library (Recipe 7) gives the company a synthetic-customer eval surface that makes daily iteration safe. The institutional memory layer (Recipe 9) gives every agent in the fleet access to the company's compiled current truth.

These work as a pair. The personas let the team move fast. The memory layer ensures every agent decision is informed by what the company already knows. With both in place, every subsequent recipe ships faster and with fewer failure modes.

Step 4: Add self-improvement loops

Recipe 8. Start with one loop targeting the highest-volume Part I agent (usually customer support). Run prompt-only self-improvement first. Once the technical owner is comfortable reviewing prompt rewrites, extend the loop to tools and code (the monitoring agent that fixes its own infrastructure, covered in Chapter 8).

At this point the company has the compounding mechanism running. Improvements happen while the team sleeps. The technical owner promotes the winners through the eval factory. The metric curves bend up week over week.

Step 5: Per-customer agents and the L4 transition

Recipe 6 enters pilot at ten thousand top-LTV users. The vector embedding architecture is live. Voice-native operations (Chapter 11) extend to the per-customer agents as a channel. Long-horizon agents (Chapter 12) handle the manual operational bottlenecks the portco named in Step 1.

In parallel, the org structure from Chapter 13 takes shape. Centralised AI team. Workstream model. Burn-tokens-not-headcount budget conversations. Middle management gets thinner. Function leads own their own evals and their own performance-review criteria around AI workflow creation. The company is at L4.

What changes after Step 5

The compounding gets visible. The eval datasets have grown across multiple iteration cycles. The synthetic persona library has been refreshed several times. Self-improvement loops have run many cycles. The institutional memory layer holds hundreds of thousands of entities. None of it is replicable by a competitor in a comparable time window, even with unlimited capital.

Garry Tan has argued the broader point: the next generation of category winners are the companies treating AI as core infrastructure rather than a feature. The five steps in this chapter are what "core infrastructure" actually means. The companies that build it are the ones that hold the moat two or three iteration cycles from now. That is why we wrote this.

Appendix

Advanced stack: tools and references

The L3-L4 tooling stack. Beyond what Part I covered. Pick within categories.

CATEGORY	OPTIONS	NOTES
Multi-agent frameworks	Anthropic Agent SDK · LangGraph · Microsoft Agent Framework · CrewAI	The Anthropic SDK is the closest match to the patterns in this book. Use it unless the team has a specific reason to deviate. AutoGen has folded into the Microsoft Agent Framework (GA 2026); the community fork continues as AG2.
Eval frameworks	Braintrust · Weights & Biases · Patronus · LangSmith · Custom Postgres	Custom Postgres is viable if the eng team has the bandwidth. Otherwise pick Braintrust or LangSmith.
Vector databases	Pinecone · Weaviate · Postgres pgvector · Qdrant	pgvector is the right choice up to a few million vectors. Above that, evaluate Pinecone or Weaviate.
Observability	Helicone · Langfuse · Arize · Custom	Langfuse is open source and self-hostable. Helicone is the fastest setup but reportedly went into maintenance mode in 2026, so favour Langfuse or Arize for new builds. Arize is the most enterprise-ready.
Orchestration	Claude Agent SDK · n8n · Temporal · Airflow	Default to Claude Agent SDK. n8n for non-engineer workflow building. Temporal when durable multi-day execution matters.
Long-horizon execution	Anthropic Computer Use · Browserbase · Browserless	Anthropic Computer Use is the most flexible. Browserbase wraps it with session management. Both work in production.

Engineering orchestration	Claude Code · GitHub MCP · Jira MCP · Linear MCP	Recipe 10 stack. Claude Code does the heavy lifting; the MCPs are the I/O surface.

Reference implementations

Code repositories the technical owner should study before building each recipe.

PATTERN	REFERENCE	RELEVANT CHAPTER
Self-improvement loop	Andrej Karpathy, autoresearch (github.com/karpathy/autoresearch)	Chapter 8
Institutional memory	Garry Tan, GBrain (github.com/garrytan/gbrain)	Chapter 9
Multi-agent research	Anthropic, multi-agent research framework (anthropic.com/engineering)	Chapter 2
Computer use	Anthropic, Computer Use API examples (docs.anthropic.com)	Chapter 14

The technical owner's steady-state cadence

Once the Part II stack is in production, the technical owner's job is operational rhythm. Five recurring checks. None of them takes long. Skipping them is how the stack rots in place.

- Each day. Check the eval dashboard. Any agent down more than 3 percentage points week-over-week gets investigated before end of day.
- Each cycle end. Review the Recipe 8 Notion page. Promote approved variants. Reject the rest with a one-line reason.
- Each cycle end. Check the Recipe 9 heartbeat log. Confirm entities-updated is non-zero.
- Each review cycle. Pull the adoption view. Share with the founder. Flag any function that has not contributed a skill in two cycles.
- As needed. Handle guardrail fires within the defined window. If the same guardrail fires twice in one cycle on the same agent, review that agent's system prompt before the next run.

Abbreviations introduced in Part II

First mentions throughout this part. Carried forward from Part I where relevant.

TERM	FULL FORM	FIRST USED
CAC:LTV	Customer Acquisition Cost to Lifetime Value ratio	Chapter 4
CI/CD	Continuous Integration / Continuous Deployment	Chapter 10
CPM	Cost Per Mille (cost per 1,000 ad impressions)	Chapter 4
CTR	Click-Through Rate	Chapter 4
DPDPA	Digital Personal Data Protection Act 2023 (India)	Chapter 3
DRI	Directly Responsible Individual	Chapter 13
HSM	Highly Structured Message (WhatsApp template)	Chapter 5
LLM-as-judge	Using a language model to score outputs from another model	Chapter 3
LTV	Lifetime Value (revenue from a customer over their lifetime)	Chapter 4
MCP	Model Context Protocol	Chapter 9
PII	Personally Identifiable Information	Chapter 2
PRD	Product Requirements Document	Chapter 10
PR	Pull Request (GitHub or similar code review)	Chapter 10
RevOps	Revenue Operations	Chapter 13
ROI	Return on Investment	Chapter 14
SDK	Software Development Kit	Chapter 2
SOC 2	Service Organization Control 2 (compliance framework for service providers)	Chapter 3
SOP	Standard Operating Procedure	Chapter 9
pgvector	Postgres extension for vector similarity search. Used in Recipes 6 and 9. Sufficient up to a few million vectors; migrate to Pinecone or Weaviate beyond that.	Chapter 6
SQS	Amazon Simple Queue Service. Managed message queue used in Recipe 6’s event-driven embedding pipeline.	Chapter 6

Source materials this part draws on

- Andrej Karpathy, "Software Is Changing (Again)," 2025. The framing on autonomy sliders, partial autonomy products, and "keep the AI on a leash" shaped Chapters 2 and 12. The autoresearch pattern in Chapter 8 is the direct reference for self-improvement loops.
- Bessemer Atlas case study on an AI-native B2B SaaS portfolio company (publicly available at bvp.com). The eval factory in Chapter 3, the outcome-based pricing model in Chapter 14, and the org design in Chapter 13 are direct lifts from this published transition.
- The Anthropic Founder's Playbook ("Building an AI-Native Startup"). The surface choices (Chat, Cowork, Code), the CLAUDE.md pattern, the Plan Mode framing, the skills-as-moat framing, and the workflow lock-in discussion are from this source.
- Faisal Mushtaq, "The 2026 VC Playbook," March 2026. The pricing analysis in Chapter 14 and the model concentration risk discussion in Chapter 2 use this source.
- Direct engagements with seven BVP India portfolio companies across consumer, e-commerce, consumer AI, B2B SaaS, and pet healthcare. Anonymised throughout as "one of our portfolio companies," "a portfolio company of ours," or descriptive variants. Field notes and borrowed-intelligence patterns throughout.

If you were building your company today,

would you start it in this shape?

Part II ends here. Questions, pushback, and field corrections welcome.

Ishan Parikh · Bessemer Venture Partners India · May 2026